



Universidad Técnica Federico Santa María
Departamento de Industrias
Asignatura: ICN-292
Profesor: José Miguel González Paul

Laboratorio 3

Automatización del triage de devoluciones en n8n

ICN-292 — Sistemas de Información para la Gestión

Estudiante: Juan Pablo Godoy
Rol: 202360577-8
Semilla personal S : 620
Umbral monetario U : \$50.000
Plazo máximo D : 7 días

Resumen ejecutivo

Se desarrolló un producto mínimo viable en n8n para recibir solicitudes de devolución de AndesHogar SpA, aplicar reglas de triage con precedencia, convertir el monto a unidades de fomento, registrar el resultado y responder al sistema emisor. La semilla $S = 620$ determina $U = \$50.000$ y $D = 7$ días.

El workflow emisor procesó las quince solicitudes entregadas. Se registraron cuatro aprobaciones, cuatro revisiones y siete rechazos, con un monto total de 30,55 UF y una tasa de aprobación de 26,67 %. El workflow programado consolida el día a las 23:59 y envía un correo. Las pruebas cubren entradas inválidas, límites inclusivos, un cambio intencional de tipo y casos que requieren validaciones adicionales.

El link del repositorio github es: <https://github.com/Stump3412/ICN292-Lab3-Godoy-JuanPablo>

21 de septiembre de 2026

1. Parámetros y alcance

1.1. Cálculo de los parámetros personales

La semilla asignada es $S = 620$. Los parámetros del proceso se calcularon según las expresiones del enunciado:

$$U = 30.000 + 1.000 \cdot (620 \bmod 50) = 50.000 \text{ CLP},$$

$$D = 7 + (620 \bmod 4) = 7 \text{ días}.$$

Cuadro 1: Parámetros utilizados por los workflows.

Parámetro	Valor	Uso
Semilla personal S	620	Identifica la versión individual del caso
Umbral monetario U	\$50.000	Separa aprobación y revisión por monto
Plazo máximo D	7 días	Determina vigencia de la solicitud

1.2. Reglas de negocio

El nodo *Switch* evalúa las reglas en el orden indicado. La primera coincidencia fija la ruta que se tomará y detiene la evaluación del ítem. La configuración mantiene desactivada la opción de enviar el dato a todas las salidas coincidentes.

Cuadro 2: Precedencia de las reglas de triage.

Orden	Ruta	Condición	Salida del Switch
1	DATOS_INVALIDOS	Falta <code>id_solicitud</code>	INVALIDO_CAMPO
2	DATOS_INVALIDOS	<code>monto</code> es menor o igual que cero	INVALIDO_MONTO
3	RECHAZO	<code>dias_desde_compra</code> es mayor que 7	RECHAZO_PLAZO
4	RECHAZO	<code>estado_producto</code> es <code>danado_por_uso</code>	RECHAZO_ESTADO
5	REVISION	<code>monto</code> es mayor que \$50.000	REVISION_MONTO
6	REVISION	<code>estado_producto</code> es <code>con_fallas</code>	REVISION_FALLA
7	APROBACION	Ninguna regla anterior se cumple	<i>Fallback Output</i>

Las comparaciones son inclusivas en los límites aceptados. Como se verá más adelante, un monto igual a U continúa hacia aprobación si no existe otra causal, y una solicitud con exactamente D días se considera dentro de plazo.

2. Parte A — Flujo de triage

2.1. Arquitectura del workflow principal

El workflow *Flujo de triage* está publicado y comienza con un Webhook POST. El cuerpo recibido queda disponible bajo `$json.body`. El nodo *Switch* clasifica cada solicitud y dirige el ítem hacia uno de los cuatro nodos *Edit Fields*. Luego, cada rama agrega los campos *ruta* y *motivo*. Las salidas convergen en la consulta del valor de la UF y en un nodo *Merge*; luego se calcula el monto en UF, se inserta una fila en la Data Table y se responde al webhook.

Cuadro 3: Nodos del workflow principal y responsabilidad.

Nodo	Tipo	Responsabilidad
Webhook	Webhook	Recibir una solicitud mediante POST
Switch	Switch	Aplicar las seis reglas y el fallback con precedencia
Inválido	Edit Fields	Asignar ruta DATOS_INVALIDOS y su motivo
Rechazo	Edit Fields	Asignar ruta RECHAZO y su motivo
Revisión	Edit Fields	Asignar ruta REVISION y su motivo
Aprobación	Edit Fields	Asignar ruta APROBACION y su motivo
Consultar valor UF	HTTP Request	Obtener indicadores desde <code>https://mindicador.cl/api</code>
Merge	Merge	Combinar la solicitud clasificada y la respuesta de la API
Valor en UF	Edit Fields	Calcular <code>monto_uf</code> e incorporar <i>U</i> y <i>D</i>
Insert row	Data Table	Registrar el resultado en la tabla <i>Tabla</i>
Respond to Webhook	Respond to Webhook	Devolver ruta y motivo al emisor

El flujo contiene once nodos y supera el mínimo de ocho indicado en el enunciado. La respuesta HTTP funciona como notificación sincrónica para el sistema que envía la solicitud. El workflow principal no incorpora un canal asíncrono adicional para el cliente, como correo o mensajería, pero si se tuviese un servidor de correo propio se podría enviar un mensaje al correo de la solicitud.

2.2. Workflow emisor

El workflow *Workflow emisor* lanza la carga de las quince solicitudes. Su secuencia es *Manual Trigger*, *Cargar solicitudes*, *Split Out* y *HTTP Request*. El nodo de carga contiene el arreglo JSON, *Split Out* crea un ítem por solicitud y el HTTP Request realiza un POST a la URL de producción del webhook. La ejecución utiliza lotes de un ítem con un intervalo de 100 ms.

Esta separación permite mantener el webhook escuchando en su workflow publicado y ejecutar el emisor desde otra pestaña. La URL de producción se usa porque permanece disponible mientras el flujo principal está activo; la URL de prueba solo acepta peticiones durante la escucha temporal iniciada desde el editor.

El siguiente objeto representa el cuerpo de una petición individual después de *Split Out*:

```
{
  "id_solicitud": "DEV-1115",
  "sku": "AU-210",
  "unidades": "1",
  "monto": "19990",
```

```
"dias_desde_compra": "3",
"estado_producto": "sin_abrir",
"email_cliente": "cliente1@example.cl"
}
```

2.3. Consulta de la UF y transformación del monto

El nodo *Consultar valor UF* realiza una petición GET sin autenticación a `https://mindicador.cl/api`. Se agregaron los encabezados `Accept: application/json` y `User-Agent: Mozilla/5.0` para obtener una respuesta válida desde la instancia local, pues no se podía acceder sin estos. La siguiente respuesta abreviada corresponde al JSON usado durante las pruebas y se incorpora porque no forma parte de los archivos entregados:

```
{
  "version": "1.7.0",
  "autor": "mindicador.cl",
  "fecha": "2026-09-16T20:00:00.000Z",
  "uf": {
    "codigo": "uf",
    "nombre": "Unidad de fomento (UF)",
    "unidad_medida": "Pesos",
    "fecha": "2026-09-16T03:00:00.000Z",
    "valor": 40942.74
  }
}
```

El monto se transforma con la expresión:

```
{{ Number(($json.body.monto.toNumber() / $json.uf.valor).toFixed(2)) }}
```

`toNumber()` convierte el monto recibido como texto a número. La división expresa el monto en UF. `toFixed(2)` redondea a dos decimales y produce un texto, por lo que `Number(...)` devuelve un valor numérico apto para sumar en el resumen diario, aunque no sea estrictamente necesario.

2.4. Registro y trazabilidad

La Data Table *Tabla* contiene `id_solicitud`, `sku`, `monto_uf`, `ruta`, `motivo`, U y D. `n8n` agrega automáticamente `id`, `createdAt` y `updatedAt`, que se utilizan para seleccionar los registros del día. En particular se usa `createdAt`, ya que es la fecha en que se emitió efectivamente la solicitud

La tabla de resultados presentada a continuación muestra el monto en pesos, los días, el estado y la clasificación observada. Además, se agregó a la entrega el CSV de esta tabla con los montos en UF.

Cuadro 4: Resultados de las quince solicitudes procesadas.

ID	SKU	Monto CLP	Días	Estado	Ruta	Motivo	UF
DEV-1115	AU-210	19.990	3	sin_abrir	APROBACION	La solicitud cumple las condiciones de aprobación	0,49
DEV-1116	TE-330	69.980	5	abierto_sin_uso	REVISION	Monto superior al umbral de \$50.000	1,71
DEV-1117	HV-050	24.990	12	con_fallas	RECHAZO	Solicitud fuera del plazo máximo de 7 días	0,61
DEV-1118	MW-150	79.990	4	sin_abrir	REVISION	Monto superior al umbral de \$50.000	1,95
DEV-1119	PL-120	45.990	25	danado_por_uso	RECHAZO	Solicitud fuera del plazo máximo de 7 días	1,12
DEV-1120	AS-260	89.990	2	con_fallas	REVISION	Monto superior al umbral de \$50.000	2,20
DEV-1121	LV-330	289.990	9	sin_abrir	RECHAZO	Solicitud fuera del plazo máximo de 7 días	7,08
DEV-1122	PL-120	45.990	2	sin_abrir	APROBACION	La solicitud cumple las condiciones de aprobación	1,12
DEV-1123	AU-210	39.980	15	danado_por_uso	RECHAZO	Solicitud fuera del plazo máximo de 7 días	0,98
DEV-1124	HV-050	49.980	1	abierto_sin_uso	APROBACION	La solicitud cumple las condiciones de aprobación	1,22
DEV-1125	TE-330	34.990	22	sin_abrir	RECHAZO	Solicitud fuera del plazo máximo de 7 días	0,85
DEV-1126	MW-150	79.990	8	con_fallas	RECHAZO	Solicitud fuera del plazo máximo de 7 días	1,95
DEV-C01	CB-010	9.990	1	sin_abrir	APROBACION	La solicitud cumple las condiciones de aprobación	0,24
DEV-C02	LV-330	289.990	2	sin_abrir	REVISION	Monto superior al umbral de \$50.000	7,08
DEV-C03	MW-150	79.990	29	abierto_sin_uso	RECHAZO	Solicitud fuera del plazo máximo de 7 días	1,95

Cuadro 5: Consolidado de los resultados registrados.

Ruta	Solicitudes	Monto total (UF)	Participación
APROBACION	4	3,07	26,67 %
REVISION	4	12,94	26,67 %
RECHAZO	7	14,54	46,67 %
DATOS_INVALIDOS	0	0,00	0,00 %
Total	15	30,55	100,00 %

Los casos DEV-1117, DEV-1119, DEV-1120, DEV-1121, DEV-1123 y DEV-1126 satisfacen más de una condición posible, pero caen en la primera ruta según el orden de la tabla 2.

3. Parte B — Flujo programado de resumen

3.1. Lectura del registro y filtro diario

El workflow *Workflow de resumen* se configuró con zona horaria *America/Santiago* y un *Schedule Trigger* diario a las 23:59. Esta hora permite consolidar las solicitudes registradas durante la jornada y evitar perder solicitudes. Para ejecutar el disparador sin intervención, el workflow debe quedar publicado y activo.

La secuencia utilizada es:

1. *Schedule Trigger* inicia la ejecución diaria.
2. *Get row(s)* lee todas las filas de la Data Table *Tabla*.
3. *Solicitudes de hoy* filtra `createdAt` desde el inicio del día hasta el inicio del día siguiente.
4. *Hubo solicitudes?* comprueba que exista un `id_solicitud`.
5. *Resumen por ruta* cuenta solicitudes y suma `monto_uf`, agrupando por `ruta`.
6. *Indicadores* calcula totales y tasa de aprobación.
7. *Preparar mensaje* construye el cuerpo del correo.
8. *Send an Email* envía un único resumen diario.

El filtro utiliza `createdAt`, un campo automático de n8n. La comparación respeta la zona horaria del workflow y evita mantener una columna manual de fecha de procesamiento.

3.2. Resumen consolidado

El nodo *Summarize* produce una cuenta y una suma por ruta. Después, el nodo *Indicadores* genera un solo ítem con las métricas solicitadas para el correo.

1 ítem		
APROBACION	REVISION	RECHAZO
count_id_solicitud : 4	count_id_solicitud : 4	count_id_solicitud : 7
sum_monto_uf : 3.0700000000000003	sum_monto_uf : 12.940000000000001	sum_monto_uf : 14.54

Figura 1: Resultado consolidado por ruta en un solo ítem.

Cuadro 6: Indicadores incorporados al mensaje diario.

Indicador	Resultado
Solicitudes aprobadas	4
Solicitudes en revisión	4
Solicitudes rechazadas	7
Solicitudes con datos inválidos	0
Monto aprobado	3,07 UF
Monto en revisión	12,94 UF
Monto rechazado	14,54 UF
Monto total del día	30,55 UF
Tasa de aprobación	26,67 %

3.3. Día sin solicitudes y envío del correo

Si el filtro no encuentra solicitudes, la rama falsa del nodo *Hubo solicitudes?* llega a *Mensaje sin solicitudes*. Ese nodo genera un texto con cero solicitudes, cero UF y tasa de aprobación de 0,00 %. Ambas ramas se conectan al mismo nodo *Send an Email*, por lo que se envía exactamente un correo por ejecución.

El nodo de correo utiliza credenciales almacenadas en el gestor de n8n. El archivo exportado y este informe no muestran contraseñas, tokens ni URL con credenciales. La prueba manual confirmó el envío del mensaje, que se adjunta en la siguiente imagen.



Figura 2: Correo recibido de forma automática.

4. Parte C — Errores y casos borde

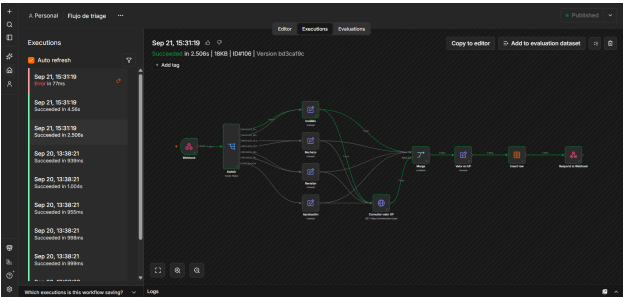
4.1. Tres entradas inválidas

Se enviaron tres entradas distintas, que se incluyen completas para dejar reproducible la prueba.

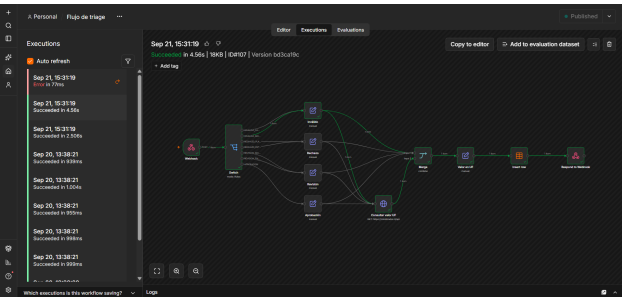
```
{
  "solicitudes": [
    {
      "sku": "Prueba1",
      "unidades": "1",
      "monto": "19990",
      "dias_desde_compra": "2",
      "estado_producto": "sin_abrir",
      "email_cliente": "prueba1@example.cl"
    },
    {
      "id_solicitud": "Prueba2",
      "sku": "PL-120",
      "unidades": "1",
      "monto": "0",
      "dias_desde_compra": "2",
      "estado_producto": "sin_abrir",
      "email_cliente": "prueba2@example.cl"
    },
    {
      "id_solicitud": "Prueba3",
      "sku": "PL-120",
      "unidades": "1",
      "monto": "19990",
      "dias_desde_compra": "ayer",
      "estado_producto": "sin_abrir",
      "email_cliente": "prueba3@example.cl"
    }
  ]
}
```

Cuadro 7: Comportamiento esperado y observado de las entradas inválidas.

Entrada	Comportamiento esperado	Comportamiento observado
Prueba 1: falta identificador	Ruta DATOS_INVALIDOS por campo obligatorio vacío	El Switch activó INVALIDO_CAMPO; la ejecución continuó por el nodo Inválido
Prueba 2: monto igual a cero	Ruta DATOS_INVALIDOS por monto menor o igual que cero	El Switch activó INVALIDO_MONTO; la ejecución continuó por el nodo Inválido
Prueba 3: días con texto ayer	Falla al convertir el valor para una comparación numérica	El Switch se detuvo con <i>cannot convert to number</i>

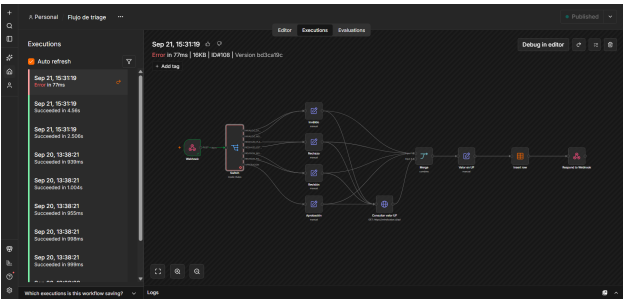


(a) Solicitud sin `id_solicitud`.

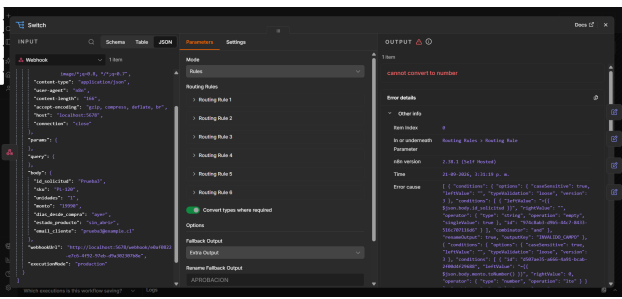


(b) Solicitud con monto igual a cero.

Figura 3: Ejecuciones clasificadas en la rama de datos inválidos.



(a) Ejecución detenida en el Switch.



(b) Detalle *cannot convert to number*.

Figura 4: Falla provocada por `dias_desde_compra = .ayer`.

4.2. Configuración On Error

Los nodos regulares de n8n presentan tres comportamientos posibles ante una falla:

Cuadro 8: Alternativas de On Error en n8n.

Valor	Efecto
Stop Workflow	Detiene la ejecución y la marca como fallida
Continue	Continúa el flujo y entrega un ítem con la información disponible
Continue (using error output)	Envía el error por una salida separada para tratarlo mediante una rama específica

En los tres archivos exportados, los nodos conservan el comportamiento predeterminado *Stop Workflow*. Se eligió para el Switch porque una conversión fallida impide aplicar correctamente la política; para la consulta de UF porque el monto transformado depende de la respuesta; para Merge, Edit Fields y Data Table porque sus salidas son necesarias para la trazabilidad y para Respond to Webhook porque una respuesta incompleta no representa una ejecución satisfactoria. En el workflow diario se mantiene la misma opción en lectura, filtro, consolidación y correo para que la pestaña *Executions* registre cualquier falla.

Una evolución recomendable consiste en configurar *Continue (using error output)* en el HTTP Request y en el envío de correo, conectar esas salidas a nodos de registro técnico y mantener una señal de error visible para soporte.

4.3. Error Workflow

Un *Error Workflow* es un workflow separado que comienza con *Error Trigger*. n8n lo ejecuta cuando otro workflow vinculado termina con error y le entrega datos como el nombre del flujo, el identificador de ejecución, el nodo que falló y el mensaje técnico. Esa información puede guardarse en una tabla y enviarse a un canal de soporte para revisión.

Un workflow programado se inicia sin la necesidad de un usuario observando el editor. Si cualquier nodo falla y no existe un Error Workflow, el error queda únicamente en el historial de ejecuciones y no se da aviso, lo que es imperativo para resolverlo rápidamente.

4.4. Casos de frontera

Los dos casos límite se construyeron con un monto exactamente igual a U y con días exactamente iguales a D :

```
{
  "solicitudes": [
    {
      "id_solicitud": "Limite1",
      "sku": "PL-120",
      "unidades": "1",
      "monto": "50000",
      "dias_desde_compra": "2",
      "estado_producto": "sin_abrir",
      "email_cliente": "prueba1@example.cl"
    },
    {
      "id_solicitud": "Limite2",
      "sku": "PL-120",
      "unidades": "1",
      "monto": "19990",
      "dias_desde_compra": "7",
      "estado_producto": "sin_abrir",

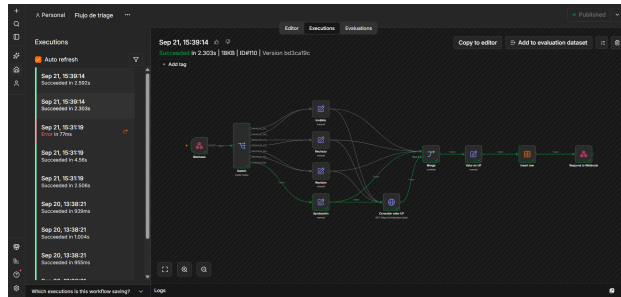
```

```

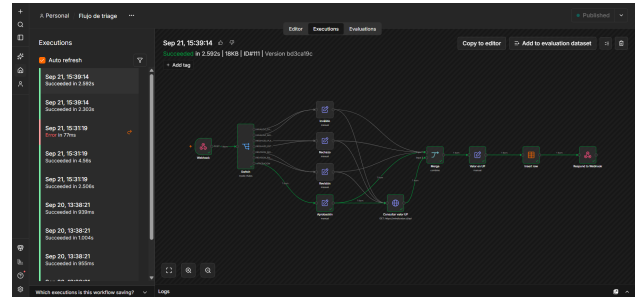
    "email_cliente": "prueba2@example.cl"
  }
]
}

```

Ambos ítems alcanzaron la salida APROBACION. La regla de revisión usa `monto >50000` y la regla de rechazo usa `dias_desde_compra >7`; las igualdades permanecen en la zona aceptada.



(a) Monto igual a \$50.000.



(b) Días desde la compra iguales a 7.

Figura 5: Ejecución satisfactoria de los dos casos de frontera.

4.5. Error de tipo provocado

Para provocar el error solicitado, la segunda condición del Switch se cambió temporalmente de tipo *Number* con operador menor o igual que cero a tipo *String* con operador *is not empty*. Se envió el siguiente cuerpo:

```

{
  "id_solicitud": "DEV-C01",
  "sku": "CB-010",
  "unidades": "1",
  "monto": "9990",
  "dias_desde_compra": "1",
  "estado_producto": "sin_abrir",
  "email_cliente": "control01@example.cl"
}

```

El monto "9990" es una cadena no vacía, por lo que la regla modificada se cumplió y la precedencia lo envió a DATOS_INVALIDOS. La condición dejó de representar la política `monto <= 0`, ya que este operador no se encontraba disponible para comparar texto.

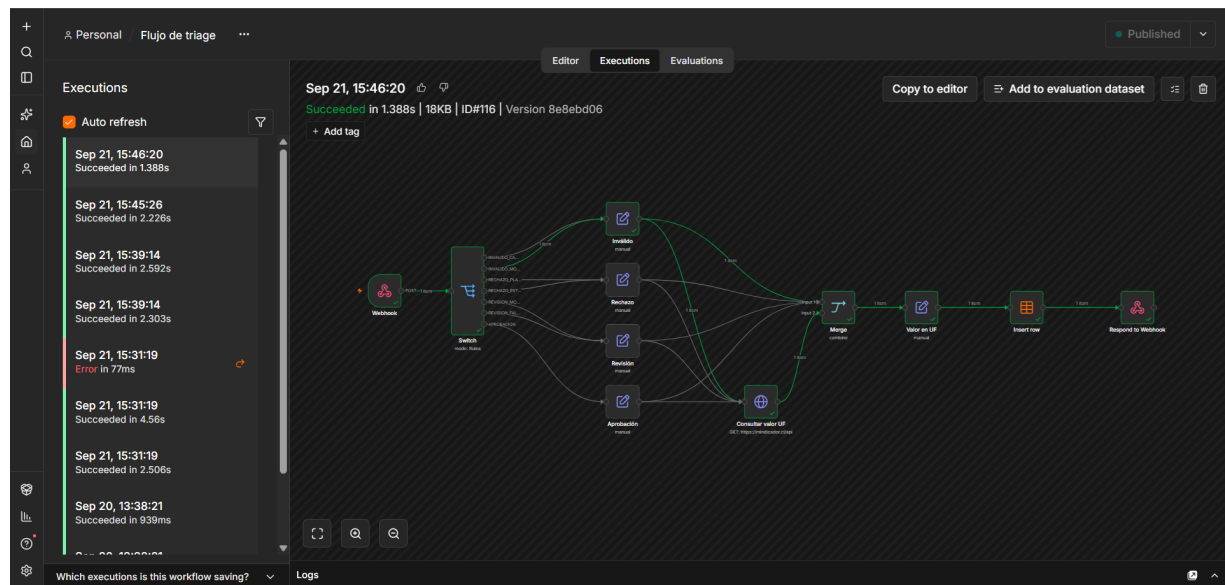


Figura 6: DEV-C01 dirigido a Inválido durante la modificación intencional del tipo.

4.6. Casos que el flujo no puede resolver

Se probaron dos solicitudes que revelan límites de validación del MVP:

```
{
  "solicitudes": [
    {
      "id_solicitud": "Limite1",
      "sku": "PL-120",
      "unidades": "1",
      "monto": "veinte mil",
      "dias_desde_compra": "2",
      "estado_producto": "sin_abrir",
      "email_cliente": "prueba1@example.cl"
    },
    {
      "id_solicitud": "Limite2",
      "sku": "ZZ-999",
      "unidades": "1",
      "monto": "19990",
      "dias_desde_compra": "3",
      "estado_producto": "sin_abrir",
      "email_cliente": "prueba2@example.cl"
    }
  ]
}
```

Monto "veinte mil". La expresión `toNumber()` no puede convertir palabras a un número y el Switch falla antes de clasificar la solicitud. El flujo necesita una validación previa que compruebe el formato numérico y envíe el ítem a `DATOS_INVALIDOS` con un motivo que sea auditable.

SKU "ZZ-999". El SKU cumple la forma de una cadena no vacía y ninguna regla consulta un catálogo de productos. La solicitud avanza como válida y llega a `APROBACION` si las demás condiciones se cumplen. Para detectarla se requiere consultar un catálogo maestro mediante Data Table, base de datos o API y tratar como inválido cualquier SKU inexistente en los registros.

5. Conclusiones

El MVP automatiza la recepción, clasificación, enriquecimiento y registro de solicitudes de devolución. La precedencia del Switch se comprobó con las quince solicitudes, y los límites $monto = U$ y $dias = D$ se comportaron según la política. El workflow emisor permite reproducir el lote completo y el resumen programado consolida las rutas, montos y tasa de aprobación en un único correo diario.

Las pruebas de la Parte C muestran que la conversión de tipos es un punto crítico. Los textos en campos numéricos producen fallas en el Switch y la ausencia de una consulta de catálogo permite aceptar SKU desconocidos. La robustez operacional requiere validaciones de esquema antes del triage y un Error Workflow que alerte sobre ejecuciones fallidas.

Antes de convertir el MVP en sistema definitivo se recomienda corregir el identificador de la respuesta del webhook, agregar un canal asíncrono de notificación al cliente si el proceso lo exige y vincular un flujo de manejo de errores. Estas mejoras completan la trazabilidad y facilitan el soporte de las ejecuciones programadas.